

Reading / Concept Check

Q1: Name the four stages a C source file passes through on its way to an executable, and what each stage takes as input and produces as output.

Preprocessing takes the raw .c source code as input. It handles things like `#include` and `#define` and removes comments. The output is the preprocessed C code. Compilation takes the preprocessed C code as input and turns it into assembly language. The output is an assembly file (.s). Assembly takes the assembly file as input and turns it into machine code. The output is an object file (.o). Linking takes the object files and any needed libraries as input. It puts everything together and produces the final executable program.

Q2: What does `-Wall` add to a gcc invocation? What does `-Wextra` add beyond `-Wall`? What does `-Werror` change about how warnings are treated?

`Wall` enables common compiler warnings for problems like unused variables and formatting mistakes. `-Wextra` adds more warnings that `-Wall` does not catch, such as unused function parameters and some sign comparison problems. `-Werror` makes GCC treat warnings as errors, so the program will not compile until the warnings are fixed.

Q3: What is the difference between compiling a file with gcc `-c` versus without `-c`?

gcc `-c` runs the source code through preprocessing, compilation, and assembly, but stops before linking. The output is an object file (.o). This is useful when you have multiple source files that you want to compile separately before linking them together. Without `-c`, GCC also does the linking step and creates the final executable program.

Q4: In a Makefile rule, what are the target, the prerequisites, and the recipe?

The target is the file that you want to create, such as an object file. The prerequisites are the files that the target depends on, such as source or header files. The recipe is the command that make runs to build the target. It is usually written on an indented line underneath the target.

Q5: Why does make usually avoid rebuilding a target that is already up to date, and how does it decide?

make checks the target and the files it depends on before rebuilding anything. If the target already exists and is newer than its prerequisites, make knows it does not need to rebuild it. If one of the prerequisite files is newer, make rebuilds the target. This helps avoid doing the same work when nothing has changed.

Q6: What is the difference between a local git commit and pushing that commit — why can a commit exist and still not count as submitted work for this course

A Git commit saves the changes in your local repository on your computer. It does not automatically send those changes anywhere else. Pushing sends the commits to the remote repository, such as your assigned GitHub or GitLab repository. For the course, the instructor needs to be able to see the work in the remote repository, so committing alone does not count as submitting it.